

Unlocking User-oriented Pages: Intention-driven Black-box Scanner for Real-world Web Applications

Weizhe Wang¹, Yao Zhang^{1*}, Kaitai Liang², Guangquan Xu^{1*}, Hongpeng Bai¹, Qingyang Yan¹, Xi Zheng⁴ and Bin Wu¹

¹Tianjin University

²Delft University of Technology

³Macquarie University

Abstract

Black-box scanners have played a significant role in detecting vulnerabilities for web applications. A key focus in current black-box scanning is increasing test coverage (*i.e.*, accessing more web pages). However, since many web applications are user-oriented, some deep pages can only be accessed through complex user interactions, which are difficult to reach by existing black-box scanners. To fill this gap, a key insight is that web pages contain a wealth of semantic information that can aid in understanding potential user intention. Based on this insight, we propose *Hoyen*, a black-box scanner that uses the Large Language Model to predict user intention and provide guidance for expanding the scanning scope. *Hoyen* has been rigorously evaluated on 12 popular open-source web applications and compared with 6 representative tools. The results demonstrate that *Hoyen* performs a comprehensive exploration of web applications, expanding the attack surface while achieving about 2× than the coverage of other scanners on average, with high request accuracy. Furthermore, *Hoyen* detected over 90% of its requests towards the core functionality of the application, detecting more vulnerabilities than other scanners, including unique vulnerabilities in well-known web applications. Our data/code is available at <https://hoyen.tjuns1.com/>

1 Introduction

Web applications have become a crucial component of the modern Internet. Detecting vulnerability in web applications is significant for maintaining cybersecurity. Due to the dynamic and complex nature of web applications, their source code is often opaque to users, making it difficult to access directly. Therefore, Black-box detection for Web applications have gained significant attention in recent years due to their ability to test without relying on specific systems or code types.

Black-box testing for web applications has been studied for many years, with a major focus on how to enhance coverage. In early research, most of the approaches rely on breadth-first search (BFS) and random navigation strategies to explore target web applications [Doupé *et al.*, 2012, Eriksson *et al.*, 2021, Pellegrino *et al.*, 2015]. Among them, [Doupé *et al.*, 2012] identified vulnerabilities by generating user input vectors based on inferred state machines. However, their heuristic-based approach faces challenges in managing complex state transitions. In light of this, [Eriksson *et al.*, 2021] built on [Pellegrino *et al.*, 2015] designed a data-driven crawler to model navigation and track inter-state dependencies. However, these methods are suboptimal for thoroughly testing web applications, as the absence of specific knowledge about the target. Consequently, they may inadvertently trigger unintended functions or actions that should remain inactive (such as logging out), leading to premature or unintended test terminations and making it difficult to detect deeper and more complex vulnerabilities.

In recent years, machine learning is extensively utilized in web application testing to optimize the efficacy of web application scanners [Hannousse *et al.*, 2024]. Traditional approaches based on machine learning [Melicher *et al.*, 2021] to detect vulnerabilities suffer from suboptimal cost-effectiveness. These approaches rely heavily on customized pre-trained models, making it difficult to dynamically adapt to the diverse and evolving nature of web applications. The introduction of the Transformer framework [Vaswani *et al.*, 2017] has led to the rise of numerous Large Language Models (LLMs). LLMs are distinguished by their powerful natural language processing and generalized reasoning abilities, consistently demonstrating superior performance in recognition and inference tasks in textual and multimodal domains [You *et al.*, 2024]. A substantial body of research has exploited the advanced capabilities of LLMs to identify software vulnerabilities and system threats [Zheng *et al.*, 2024, Jiang *et al.*, 2024, Xia *et al.*, 2024, Asmita *et al.*, 2024, Wang *et al.*, 2024]. However, the ability of LLMs to detect vulnerabilities in web applications remains constrained due to the immense scale of web applications and the complex interrelations between their various states.

Although existing researchers have made efforts to improve the coverage of black-box web testing, they have yet to consider it from the perspective of user interaction. In fact,

*Corresponding author

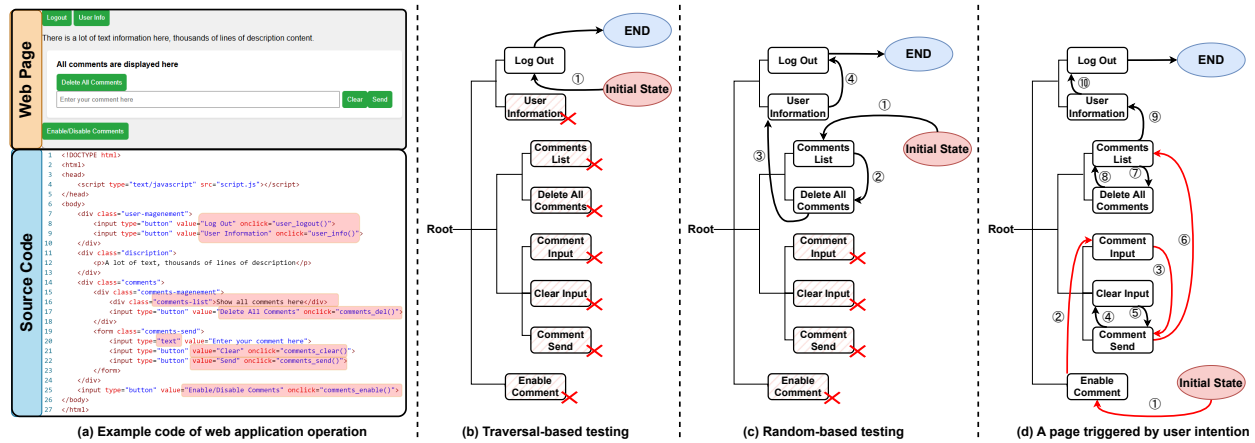


Figure 1: Motivation example. The four sub-diagrams represent the web application code with interface content, and request flow for the traversal, random, and proposed strategies. The red line indicates the flow of the actual test that triggers the vulnerability.

most contemporary web applications are typically designed around user interactions, which makes many deep-level web pages can only be reached through specific user operations. These pages pose a significant hurdle for existing black-box testing methods. We take Figure 1 as an example, where Figure 1-a shows a web page along with its source code, illustrating the commenting functionality of the web page with several functions, *e.g.*, “Comment Input”, “Comment Send”, “Clear Input”, *et al.* Testing the commenting functionality requires enabling the comment feature, inputting a comment, and submitting it, and the comment should then appear in the list. It is crucial to avoid actions like “log out” or “delete all comments”, which can disrupt the testing process. We take two state-of-the-art tools, *i.e.*, [Chaitin, 2024] and [Eriksson *et al.*, 2021], as examples to show why existing works struggle in this case. As a traversal-based tool, [Chaitin, 2024] will always trigger “log out” as soon as it begins to explore the web app, failing to reach other functions (step ① in Figure 1-b). [Eriksson *et al.*, 2021] employs a randomized strategy, allowing it to test more functions. However, due to the lack of contextual understanding, it fails to prioritize enabling the commenting functionality, preventing comment submission for testing. It also prematurely triggers actions like comment deletion and logout (① to ④ in Figure 1-c), causing early test termination. Fortunately, as illustrated in the code of Figure 1-a, web pages contain rich semantic information (highlighted in red) that can assist in predicting user intent. Figure 1 shows how a user interacts with the comment function. Specifically, the user first inputs a comment (① to ②), then either clears the input (③) or sends it to trigger a deeper page (④ to ⑩). Alternatively, the user may intend to delete the comment after it appears in the comment list (⑦ to ⑧). Clearly, understanding user intent can help identify the path to the deeper page.

From the above example, it can be seen that semantic information in web pages can infer potential user intentions, thereby helping to identify in-depth pages. In recent years, advancements in LLMs have opened new opportunities for extracting semantic data from code. Consequently, a nat-

ural idea is to leverage LLMs to analyze semantic information from web pages. Nevertheless, there are still some challenges that need to be addressed. **C1: The analysis for large-scale web page.** Real-world web applications always contain a vast amount of content. Dealing with the web page with thousands of lines of code presents a significant challenge for LLMs, which struggle with token limitation and performance degradation over extended content. Besides, the vast amount of content can also make it difficult for large models to focus on key information, leading to misjudgments by the models. **C2: Intricate dependencies in real-world scenarios.** In real-world web applications, interdependencies can significantly impact the application’s state [Doupé *et al.*, 2012, Eriksson *et al.*, 2021]. For instance, in Figure 1-c, the failure to recognize the dependency between enabling and the “Comment Send” action led to triggering the wrong action prematurely. As web applications continue to grow in size, the dependencies become increasingly complex, making their analysis more challenging.

To mitigate above challenges, we introduce an intention-driven black-box scanner for web applications, named *Hoyen*. Specifically, to address C1, we extract semantically relevant information from web pages and analyze the structure information between the page elements, ensuring efficient page analysis by LLM. To address C2, we leverage LLM, understanding the intention path and element context based on the real user intention, to infer operations that follow the user’s intent and handle dependencies in the application.

In summary, this paper makes the following contributions.

- We introduced *Hoyen*, a novel black-box scanner for web applications that employs contextual semantics and content awareness to test web applications.
- We proposed a semantic and content-awareness-based user intention prediction approach to address the complexity of web application dependencies and the limitations of conventional access strategies in exploring deeper application pages. Additionally, we proposed a function-based page content refinement approach to ex-

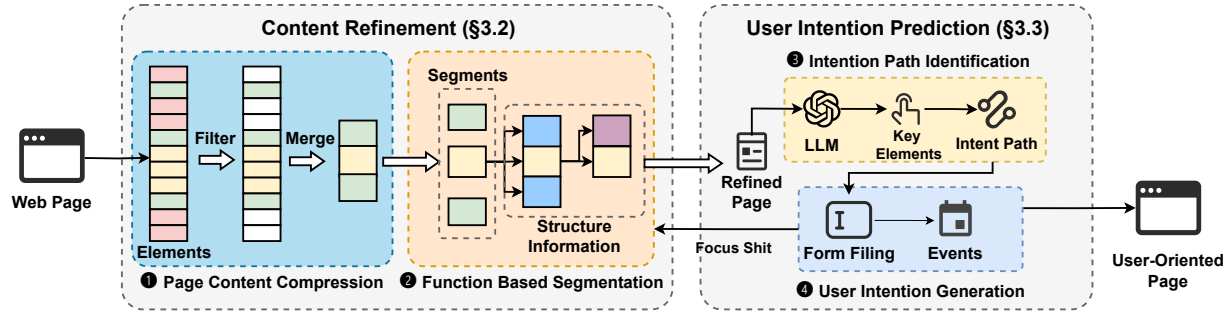


Figure 2: The framework of *Hoyen*

tract key semantics and understand the extensive information of large-scale web application pages.

- We conducted a comprehensive comparative analysis of *Hoyen* versus 6 other academic and industrial scanners in 12 different web applications. Our experimental results demonstrate that *Hoyen* exhibits superior page coverage and request accuracy compared to other methods while uncovering more vulnerabilities.

2 Related Work

[Doupé *et al.*, 2012] proposed Enemy of the State, a black-box scanner that identifies security vulnerabilities by inferring state machines but struggles to handle complex states. To address challenges posed by JavaScript-based applications, [Pellegrino *et al.*, 2015] introduced jāk, a dynamic crawler leveraging client-side JavaScript analysis. Building on this, [Eriksson *et al.*, 2021] developed BlackWidow, a data-driven scanner that enhances code coverage but remains susceptible to state loss due to incorrect edge traversal. [Zheng *et al.*, 2021] and [Zhang *et al.*, 2022] applied curiosity-driven reinforcement learning to enable efficient and adaptive exploration for automatic web testing. Still, their approaches lacked semantic understanding, limiting their ability to capture interdependency correlations within web applications. Meanwhile, [Kirchner *et al.*, 2024] focused on blind XSS detection using a polyglot methodology but failed to account for broader semantic relationships.

In contrast with black-box testing, white-box testing with direct access to source code, allows deeper analysis and vulnerability detection, such as through machine learning for XSS detection [Kaur *et al.*, 2023] and taint analysis [Luo *et al.*, 2022, Güler *et al.*, 2024, Fioraldi *et al.*, 2020], which has proven effective in identifying vulnerabilities. However white-box testing is often impractical for many systems due to language dependencies and strict requirements. Gray-box testing offers partial system insight and balances these approaches [Olsson *et al.*, 2024]. Although white-box and gray-box testing offer some advantages in the detection of vulnerabilities in web applications, black-box testing remains the preferred method due to its wide applicability [Trickel *et al.*, 2023, Wahaibi *et al.*, 2023, Zhao *et al.*, 2023]. However, the complexity of current black-box approaches limits the widespread of AI techniques in black-box vulnerability detection for web applications.

3 Approach

3.1 Overview

Black-box testing is a widely used method for identifying vulnerabilities in web applications. Its main advantage is that it does not necessitate a comprehensive understanding of the target system, eliminating the need for system-specific adaptations. Thus, it has garnered considerable attention and adoption in academic and industrial contexts.

Figure 2 shows the overview of *Hoyen*, which consists of two component: Content Refinement (Section 3.2) and User intention Prediction (Section 3.3).

Content Refinement. Given a real-world web page, *Hoyen* first compress the page to eliminate the elements which irrelevant to semantics and merge the similar elements to reduce the content scale (See ❶). Then *Hoyen* segments the web page into specific functional block to provide structure information of pages for LLM (See ❷).

User Intention Prediction. After refining the page content, *Hoyen* identify the key elements associate with page action to infer the possible user intent (see ❸). Then, *Hoyen* generates user intentions by understanding the dependence between different elements (see ❹).

3.2 Content Refinement

For large real-world web applications, we first reduce the content of the page. This section consists of two parts, *i.e.*, page content compression and function based segmentation.

Page Content Compression

To compress the page content, our key idea is to eliminate semantically irrelevant information and merge similar pages, thereby reducing the overall page size.

We begin by organizing all major HTML tags and functions, and classify elements into three categories based on their attributes: 1) *Style elements*, used for front-end customization like CSS styles; 2) *association elements*, which are semantically related consecutive elements, such as `<p>` and `<br>`; and 3) *core functional elements*, supporting core application functions, such as forms. Since style elements are irrelevant to semantic understanding, we use a matching-based method to traverse and remove them. Association elements, such as continuous long paragraphs of text content, which often appear in sequence with similar semantics, are processed by focusing solely on their value and content, then

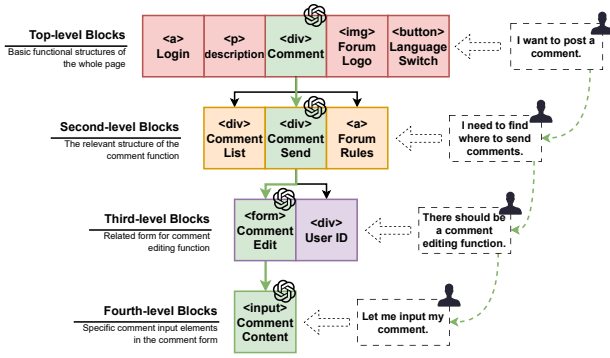


Figure 3: Example of function segmentation. Each lattice comprises structure tags and semantic information. This page is recursively segmented into multiple layers of structure, starting with the comment function, and culminating in the comment input element.

merging and abbreviating their content with semantics. For core functional elements, we recursively examine their content and child elements, eliminate irrelevant content, retaining only relevant functional and semantic information.

After eliminating irrelevant information from the pages, we merge similar pages that may widely exist in the application. To this end, we use a multidimensional similarity measure to evaluate their similarity. We first check whether the URLs of two pages adhere to the same-origin policy. Based on this, we use the LCS algorithm to comprehensively measure the similarity of the URLs, request paths, and query parameters, ensuring they meet predefined similarity thresholds. Given the dynamic nature of page rendering, URL similarity alone cannot fully capture the similarity relationship between pages. Therefore, we also analyze page-specific HTML information. Since the DOM structure reflects functional features and the style indicates visual features, we evaluate their similarity comprehensively. Specifically, we use Tree Edit Distance to compare DOM structure similarity, and Jaccard Similarity for style comparison, and combine these measures with predefined weights for overall similarity.

Function Based Segmentation

To help LLM understand the content, we segment the HTML pages based on functionality. The key insight comes from the hierarchical structure of the HTML DOM tree. Specifically, the related functions are generally grouped within the same branch, while adjacent functions and elements tend to be located at the same level of the tree. Parent-child relationships and branching patterns of the tree reflect the dependencies between elements. Leveraging these characteristics, we are able to extract functionally and semantically relevant small-scale blocks.

We use the case shown in Figure 3 to illustrate the segmentation process. Starting from the top level of the HTML DOM tree, we divide the page into five functional blocks. For example, the block “<div> Comment” represents the comment function. If the scanner focuses on this block, it further segments it into smaller blocks based on their functions. In this case, the “<div> Comment” block is recursively divided into three sub-blocks, *i.e.*, “<div> Comment list”, “<div>

Comment sent”, and “<div> Rules”, each of which serves a specific function. By recursively segmenting each level, we eventually obtain the smallest block containing semantic information, namely, “<div> Comment content”. Thus we can get contextual information from top-level to low-level to help LLM better understand user intention.

3.3 User intention Prediction

To complex dependencies in web applications, we propose user intention prediction to guide web application exploration. This section is divided into two parts, *i.e.*, intention path identification and user intention generation.

Intention Path Identification

When a user selects a specific function in an application, their decision follows a certain logical thought process. On the application interface, the dependencies between elements create a sequence that reflects the user’s intent. This sequence could form an intention path, guiding users toward accessing specific functions or user-oriented pages.

Algorithm 1 Key Elements Identification

Input: Web application page information, *Data_page*

Output: Key elements of the current page

```

1: Page_blocks  $\leftarrow$  page_divided(Data_page)
2: repeat
3:   Data_blocks  $\leftarrow$  data_extract(Page_blocks)
4:   Block_select  $\leftarrow$  select_block(Data_blocks)
5:   if scale(Page_blocks)  $\leq$  threshold then
6:     Elms  $\leftarrow$  getValidElms(Block_select)
7:   else
8:     Page_blocks  $\leftarrow$  Block_select
9:   end if
10: until Elms  $\neq$  NULL
11: Elms_Info  $\leftarrow$  get_elms_info(Elms)
12: Elem_key  $\leftarrow$  select_elem(Elms, Elms_Info)
13: return Elem_key

```

Our user intention path identification approach models the interaction and reasoning process of real users. Initially, we analyze the application structure and semantics from the function based segmentation approach (see Section 3.2). Based on this, we construct the prompt guided by the user intention and operation logic to drive LLM. By analyzing the overall context and the functional semantics of each block, we identify the block that best aligns with the user’s intent. The user intention path is then constructed by combining the application’s functional block, and progressively refining the intent from top to bottom. Through recursive traversal of the intention path and DOM structure, the scope is gradually narrowed, ultimately identifying the key elements that align with the user’s intent for deep interaction and exploration within the web application. Taking Figure 3 as an example, by analyzing the semantics of the web application’s top-level blocks, the user may be potentially interested in the comment function and would like to post a comment. We then recursively examine the sub-structures, ultimately locating the specific comment input element, *i.e.* the key element.

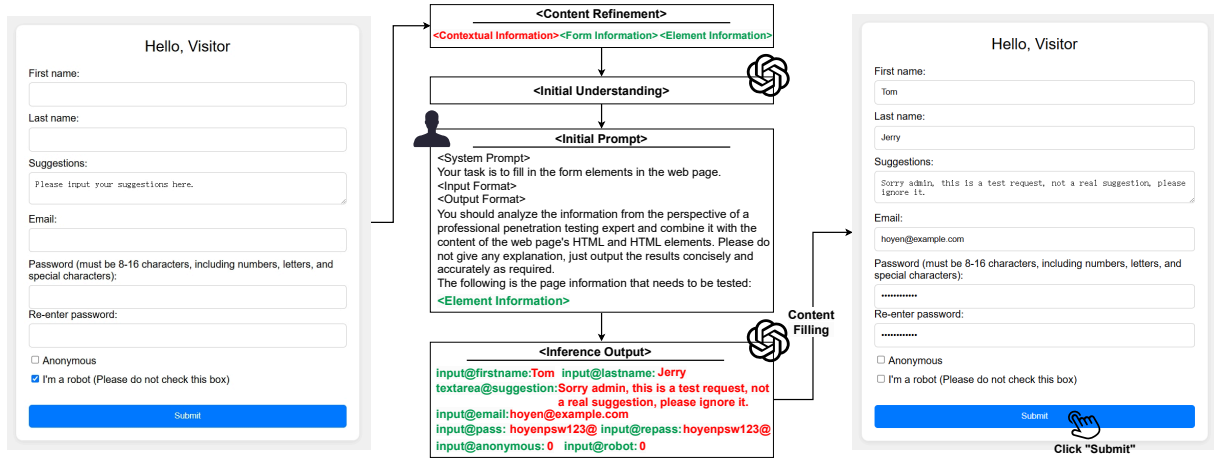


Figure 4: Example of user intention generation to the web application.

Throughout this process, the LLM drives the user intention, constructing and identifying the intention path layer by layer along the DOM tree, enabling the precise execution of the comment posting function.

Our approach to identifying key elements is shown in the Algorithm 1. First, we segment the page information based on function segmentation (Section 3.2) and extract data from specific segments *Page.blocks*. Then, the LLM makes selection decisions for each block based on user intention (Line 4). The selected functional blocks of the DOM tree are used as inputs for subsequent extractions, and the user intention path is constructed through recursive looping until the selected block meets a predefined threshold (Less than 1/20 of the max LLM context token window size). At this point, relevant elements are extracted. Finally, by analyzing the semantics and context information of these elements, the LLM selects the key elements that align with the user’s intention (Line 12).

User Intention Generation

After identifying the user intention path, further interactions with the application’s elements are needed for in-depth exploration. In real-world web applications, different elements offer various interaction methods, and often with dependencies between them. For instance, in Figure 4, On the left web page, if we check the “Anonymous” element, the “First name” and “Last name” will be invalid, demonstrating their dependence. Operations on an element typically include setting values, triggering events, etc., and are closely tied to the element’s semantics and context. From the user’s perspective, inter-element dependencies can be understood through contextual content, enabling the generation of operation logic based on their intent. In light of this, we propose an approach for user intention generation, which combines contextual information with user operations, enabling complex application interactions, to address complex inter-element dependencies.

Our approach uses LLM to understand user intent by considering prior operations and the overall application semantics, guiding decisions for subsequent actions. In particular, we would select appropriate contextual information for each element, prompts are then generated using contextual infor-

mation and page content or request scenarios. During element operations, previously filled content, context, and user intention path are passed to the LLM, which integrates them to complete the tasks. To ensure consistency, prompts for all form elements are constructed uniformly and submitted to the LLM, enabling holistic processing. For nested elements, a tree-based approach is applied to perform a recursive analysis of the content within each layer to ensure precise operations. Specifically, the entire form is inserted into the prompt firstly. Then, each element that requires individual filling is extracted and analyzed, proceeding sequentially through the form. Each element’s contextual information is examined individually, and the LLM follows the user intent to generate the content needed for interaction. Finally, the operations same as user intention are performed on the Web application.

During the test, *Hoyen* would observe and check the state changes in the web application and trace back to check if user intention and related actions are in the right direction. This process will continue until all events and elements are tested. Finally, the taint will be propagated along the application state paths to determine if there may be vulnerabilities in the application. The LLM emulates real user behavior by user intention generation, and minimizes erroneous element access or state modifications, allowing the scanner to understand inter-element dependencies and conduct comprehensive and accurate testing of the target web application.

4 Evaluation

4.1 Experimental Settings

We implemented *Hoyen* by Python which consists of over 5,000 lines¹. To evaluate the coverage extent of the scanner, we gather comprehensive access data on the server side and perform comparative analyses with other scanners.

XSS (cross-site scripting) exploits malicious HTML and JavaScript injection, requiring precise interactions with web applications and constructing an adequate exploit chain to

¹Further details can be found on our website <https://hoyen.tjunsl.com/>

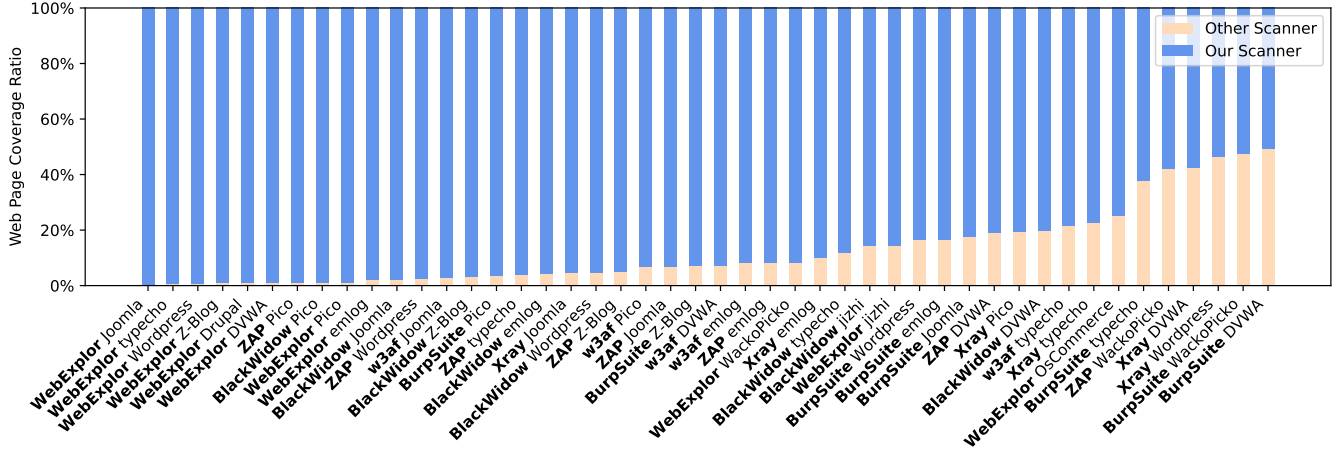


Figure 5: A comparison of page coverage between our scanner, *Hoyen*, and other scanners. Each bar is divided into two segments: one showing the page coverage achieved by *Hoyen*, and the other showing the coverage achieved by the other scanner for the same web application.

trigger vulnerabilities. Since web application operations are highly sequential and rely on dynamic feedback, effective vulnerability exploitation methods demand real-time adjustments—a challenge for scanners but intuitive for human testers. Given the representativeness of XSS, we focused our verification on XSS vulnerabilities. To assess scanner effectiveness, we reviewed vulnerability reports from various scanners for the same web application and manually reproduced the reported vulnerabilities for validation.

Approaches under comparison. We compare our scanner, *Hoyen*, with the following scanners, which are widely used in both academic research and industry: [Eriksson *et al.*, 2021], [Zheng *et al.*, 2021], [Riancho, 2024], [OWASP, 2024], [PortSwigger, 2024], and [Chaitin, 2024]. The target web application was scanned in a consistent environment using the default configurations of each scanner.

Benchmark. We tested 12 widely used open-source web applications, each with over 100 GitHub stars and an average of 15,000 stars, reflecting real-world adoption. The applications were deployed on separate Docker containers to ensure isolation and consistency in testing results.

Large Language Models. Our approach focuses on proposing a method rather than comparing LLMs performance. We tested various LLMs, including ChatGPT [OpenAI, 2024] and Gemini [Google, 2024], and found that even basic LLMs meeting the criteria significantly improve efficiency compared to traditional methods. Thus, we do not detail the impact of specific LLMs. For subsequent experiments, we primarily use the Google Gemini 1.0 API as the main LLM driver.

4.2 Coverage

As shown in Figure 5, the comparison of page coverage across different scanners for various web applications reveals that *Hoyen*, exhibits a substantial improvement in page coverage compared to other scanners, with some applications showing over a 100% increase in coverage. The coverage

results from the comparative analysis of our scanner, *Hoyen*, alongside BurpSuite and BlackWidow—currently among the most popular and state-of-the-art tools in both industry and academia are presented in Table 1.

	Hoyen	BurpSuite	BlackWidow
WordPress	121	24	6
OsCommerce	3	3	3
Prestashop	1	8	1
Joomla	222	47	5
WackoPicko	11	8	13
Drupal	93	98	198
Pico	84	3	1
typecho	143	87	19
emlog	45	9	2
Z-Blog	119	9	4
jizhi	6	23	1
DVWA	90	87	22

Table 1: The page number covered by *Hoyen*, BurpSuite, and BlackWidow across various web applications on the server.

	Hoyen	BurpSuite	BlackWidow
Wordpress	0.99	0.78	0.95
OsCommerce	1.00	0.70	1.00
Prestashop	1.00	0.88	1.00
Joomla	1.00	0.73	1.00
WackoPicko	0.99	0.72	0.61
Drupal	1.00	0.64	1.00
Pico	1.00	0.30	0.98
typecho	1.00	0.84	1.00
emlog	1.00	0.67	1.00
Z-Blog	0.94	0.93	0.99
DVWA	0.88	0.86	0.99

Table 2: The rate of successful requests for *Hoyen*, BurpSuite, and BlackWidow to all their requests.

	General functions				Core functions		Others
	Homepage	Login	Setup	About	Vulnerable	Normal	
Hoyen	1.40%	2.00%	0.00%	1.60%	58.90%	36.00%	0.10%
w3af	10.30%	24.40%	2.60%	1.30%	0.00%	0.00%	61.50%
Xray	9.40%	11.80%	2.30%	2.20%	1.50%	35.30%	37.50%
ZAP	2.40%	40.50%	2.40%	2.40%	9.60%	40.70%	2.40%
BurpSuite	6.00%	9.30%	3.10%	0.50%	17.70%	60.60%	2.70%
BlackWidow	1.50%	65.20%	3.90%	2.80%	12.00%	14.60%	0.00%

Table 3: The table of requests from various scanners regarding the different functionality of DVWA. Each cell represents the percentage of all requests for this function.

Using BlackWidow as a baseline, Hoyen demonstrated comparable or superior performance in 10 out of 12 web applications, achieving a 100% increase in overall average page coverage. For globally popular platforms like WordPress and Joomla [W3Techs, 2024], Hoyen identified 20 times more new pages than BlackWidow. On smaller applications like DVWA, Hoyen achieved a four-fold improvement in page coverage. Similarly, compared to BurpSuite, Hoyen outperformed in 8 out of 12 web applications. BlackWidow excelled on WackoPicko and Drupal due to its conservative page similarity merging strategy, leading to repeated scans of similar pages. For example, in our Drupal environment with a language-switching feature, BlackWidow rescanned nearly identical pages in different languages, achieving more page coverage. In contrast, our approach avoids redundant scans by filtering similar pages based on comparisons. On WackoPicko, BlackWidow tested uploaded file content extensively, whereas our approach prioritized XSS vulnerability detection, placing less emphasis on file upload functions. BurpSuite outperformed on PrestaShop, leveraging a dictionary-based method to discover sensitive directories and access additional pages. Our approach, lacking dictionary or brute-force techniques, may miss some web application entry points due to incomplete target system information.

4.3 Effectivity

Scanners employ varying strategies for path detection, aiming to access as many web application pages as possible for comprehensive testing. Standard methods typically start at the home page and analyze links sequentially or randomly. Some scanners may use dictionary-based brute force or enumeration, which can disrupt normal service operations. While academic research avoids brute force, scanners often generate numerous invalid requests that deviate from human navigation patterns [Jueckstock *et al.*, 2021, Li *et al.*, 2023]. To address this, we incorporate request accuracy into our evaluation, ensuring the scanner focuses on critical functions while maintaining high accuracy. Higher error rates in requests of often distinguish scanners from human users [Li *et al.*, 2023].

As shown in Table 2, our approach achieves nearly 100% successful request rates. This demonstrates that our requests efficiently access web application functions. Our approach to vulnerability detection in web applications follows a strategy similar to BlackWidow’s, starting with small-scale testing during web application modeling and crawling, followed by comprehensive vulnerability assessment at identified edges.

Both approaches generate fewer erroneous requests compared to other scanners. In our case, erroneous requests primarily occur during vulnerability testing. This is due to our broader efforts to identify additional vulnerabilities, which require more testing requests. The higher number of erroneous requests in DVWA and Z-Blog is linked to detecting more pages than BlackWidow, resulting in more erroneous requests of vulnerability tests. Despite this, our scanning strategy maintains high accuracy.

As shown in Table 3, different scanners prioritize different aspects of web applications. For example, w3af only focused on a small portion of the core functionality. BlackWidow tested core functionality and identified vulnerabilities but concentrated significantly on the login section, potentially diverting attention from the core functions. BurpSuite effectively allocated 78% of its efforts to core functions, but its scans were more dispersed due to limited page comprehension. Our approach, emphasizing XSS vulnerability detection, dedicated over 30% of requests to XSS tests, and an unexpected function detected an XSS vulnerability, accounting for nearly 25% of the total request attention. Overall, our approach proved highly effective, focusing nearly 95% of its attention on the web application’s core functionality, which was key to vulnerability identification. Based on experimental verification, we identified more vulnerabilities than other approaches. In addition to the vulnerabilities detected by existing tools, we discovered 4 unique vulnerabilities.

5 Conclusion

In response to the limitations of contemporary black-box web application scanners, this paper introduces *Hoyen*, a novel contextual semantics and content awareness black-box web application scanning approach driven by the LLMs. The approach involves identifying and selecting elements on web pages through a function-based page content refinement approach, and semantic and content-awareness-based user intention prediction approach. These approaches allow a deeper understanding of the web application’s structure and elements, enabling a more comprehensive application analysis. In our experiments, *Hoyen* demonstrated a 2× increase in average page coverage. The test requests were precisely directed toward the core functions of web applications, and identifying more vulnerabilities. Across all performance metrics, *Hoyen* outperformed existing black-box scanners in web applications.

Ethical Statement

To address potential legal and ethical concerns associated with the experiment on web scanners, we referenced the scanning red lines outlined in [Hantke *et al.*, 2024], constructed the target system in our laboratory, and conducted comparative experiments involving our scanner and other related works. Additionally, we followed ethical guidance from [Demir *et al.*, 2022] to ensure and regulate the proper conduct of our scanner. Our scanner also appends a unique tag to all User-Agent requests for detection monitoring on the server side. Throughout the evaluation process, we ensured that our experiments had minimal impact on the system. To maintain consistency in experimental results [Jueckstock *et al.*, 2021], all tests were performed on servers located in our laboratory, with all test environments locally deployed.

References

- [Asmita *et al.*, 2024] Asmita, Yaroslav Oliinyk, Michael Scott, Ryan Tsang, Chongzhou Fang, and Houman Homayoun. Fuzzing BusyBox: Leveraging LLM and crash reuse for embedded bug unearthing. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 883–900, Philadelphia, PA, August 2024. USENIX Association.
- [Chaitin, 2024] Chaitin. Xray - a powerful security assessment tool, 2024.
- [Demir *et al.*, 2022] Nurullah Demir, Matteo Große-Kampmann, Tobias Urban, Christian Wressnegger, Thorsten Holz, and Norbert Pohlmann. Reproducibility and replicability of web measurement studies. In *Proceedings of the ACM Web Conference 2022, WWW '22*, page 533–544, New York, NY, USA, 2022. Association for Computing Machinery.
- [Doupé *et al.*, 2012] Adam Doupé, Ludovico Cavedon, Christopher Kruegel, and Giovanni Vigna. Enemy of the state: A State-Aware Black-Box web vulnerability scanner. In *21st USENIX Security Symposium (USENIX Security 12)*, pages 523–538, Bellevue, WA, August 2012. USENIX Association.
- [Eriksson *et al.*, 2021] Benjamin Eriksson, Giancarlo Pellegrino, and Andrei Sabelfeld. Black widow: Blackbox data-driven web scanning. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 1125–1142, 2021.
- [Fioraldi *et al.*, 2020] Andrea Fioraldi, Dominik Maier, Heiko Eißfeldt, and Marc Heuse. AFL++ : Combining incremental steps of fuzzing research. In *14th USENIX Workshop on Offensive Technologies (WOOT 20)*. USENIX Association, August 2020.
- [Google, 2024] Google. Gemini - google deepmind, 2024.
- [Güler *et al.*, 2024] Emre Güler, Sergej Schumilo, Moritz Schloegel, Nils Bars, Philipp Görz, Xinyi Xu, Cemal Kaygusuz, and Thorsten Holz. Atropos: Effective fuzzing of web applications for Server-Side vulnerabilities. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 4765–4782, Philadelphia, PA, August 2024. USENIX Association.
- [Hannousse *et al.*, 2024] Abdelhakim Hannousse, Salima Yahiouche, and Mohamed Cherif Nait-Hamoud. Twenty-two years since revealing cross-site scripting attacks: A systematic mapping and a comprehensive survey. *Computer Science Review*, 52:100634, 2024.
- [Hantke *et al.*, 2024] Florian Hantke, Sebastian Roth, Rafael Mrowczynski, Christine Utz, and Ben Stock. Where are the red lines? towards ethical server-side scans in security and privacy research. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 4405–4423, 2024.
- [Jiang *et al.*, 2024] Yu Jiang, Jie Liang, Fuchen Ma, Yuanliang Chen, Chijin Zhou, Yuheng Shen, Zhiyong Wu, Jingzhou Fu, Mingzhe Wang, Shanshan Li, and Quan Zhang. When fuzzing meets llms: Challenges and opportunities. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering, FSE 2024*, page 492–496, New York, NY, USA, 2024. Association for Computing Machinery.
- [Jueckstock *et al.*, 2021] Jordan Jueckstock, Shaown Sarker, Peter Snyder, Aidan Beggs, Panagiotis Papadopoulos, Matteo Varvello, Benjamin Livshits, and Alexandros Kapravelos. Towards realistic and reproducible web crawl measurements. In *Proceedings of the Web Conference 2021, WWW '21*, page 80–91, New York, NY, USA, 2021. Association for Computing Machinery.
- [Kaur *et al.*, 2023] Jasleen Kaur, Urvashi Garg, and Gourav Bathla. Detection of cross-site scripting (xss) attacks using machine learning techniques: a review. *Artificial Intelligence Review*, 56(11):12725–12769, 2023.
- [Kirchner *et al.*, 2024] Robin Kirchner, Jonas Möller, Marius Musch, David Klein, Konrad Rieck, and Martin Johns. Dancer in the dark: Synthesizing and evaluating polyglots for blind Cross-Site scripting. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 6723–6740, Philadelphia, PA, August 2024. USENIX Association.
- [Li *et al.*, 2023] Xigao Li, Babak Amin Azad, Amir Rahmati, and Nick Nikiforakis. Scan me if you can: Understanding and detecting unwanted vulnerability scanning. In *Proceedings of the ACM Web Conference 2023, WWW '23*, page 2284–2294, New York, NY, USA, 2023. Association for Computing Machinery.
- [Luo *et al.*, 2022] Changhua Luo, Penghui Li, and Wei Meng. Tchecker: Precise static inter-procedural analysis for detecting taint-style vulnerabilities in php applications. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS '22*, page 2175–2188, New York, NY, USA, 2022. Association for Computing Machinery.
- [Melicher *et al.*, 2021] William Melicher, Clement Fung, Lujo Bauer, and Limin Jia. Towards a lightweight, hybrid approach for detecting dom xss vulnerabilities with machine learning. In *Proceedings of the Web Conference 2021*, pages 2684–2695, 2021.
- [Olsson *et al.*, 2024] Eric Olsson, Benjamin Eriksson, Adam Doupé, and Andrei Sabelfeld. Spider-Scents: Grey-box database-aware web scanning for stored XSS. In

- 33rd *USENIX Security Symposium (USENIX Security 24)*, pages 6741–6758, Philadelphia, PA, August 2024. USENIX Association.
- [OpenAI, 2024] OpenAI. Chatgpt - openai, 2024.
- [OWASP, 2024] OWASP. Zed attack proxy (zap), 2024.
- [Pellegrino *et al.*, 2015] Giancarlo Pellegrino, Constantin Tschürtz, Eric Bodden, and Christian Rossow. jak: Using dynamic analysis to crawl and test modern web applications. In Herbert Bos, Fabian Monrose, and Gregory Blanc, editors, *Research in Attacks, Intrusions, and Defenses*, pages 295–316, Cham, 2015. Springer International Publishing.
- [PortSwigger, 2024] PortSwigger. Burp suite - application security testing software - portswigger, 2024.
- [Riancho, 2024] Andres Riancho. w3af: web application attack and audit framework, the open source web vulnerability scanner., 2024.
- [Trickel *et al.*, 2023] Erik Trickel, Fabio Pagani, Chang Zhu, Lukas Dresel, Giovanni Vigna, Christopher Kruegel, Ruoyu Wang, Tiffany Bao, Yan Shoshitaishvili, and Adam Doupe. Toss a fault to your witcher: Applying grey-box coverage-guided mutational fuzzing to detect sql and command injection vulnerabilities. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 2658–2675, 2023.
- [Vaswani *et al.*, 2017] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *CoRR*, abs/1706.03762, 2017.
- [W3Techs, 2024] W3Techs. Usage statistics and market shares of content management systems, 2024.
- [Wahaibi *et al.*, 2023] Salim Al Wahaibi, Myles Foley, and Sergio Maffei. SQIRL: Grey-Box detection of SQL injection vulnerabilities using reinforcement learning. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 6097–6114, Anaheim, CA, August 2023. USENIX Association.
- [Wang *et al.*, 2024] J. Wang, L. Yu, and X. Luo. Llmif: Augmented large language model for fuzzing iot devices. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 881–896, Los Alamitos, CA, USA, may 2024. IEEE Computer Society.
- [Xia *et al.*, 2024] Chunqiu Steven Xia, Matteo Paltenghi, Jia Le Tian, Michael Pradel, and Lingming Zhang. Fuzz4all: Universal fuzzing with large language models. ICSE ’24, New York, NY, USA, 2024. Association for Computing Machinery.
- [You *et al.*, 2024] Keen You, Haotian Zhang, Eldon Schoop, Floris Weers, Amanda Swearngin, Jeffrey Nichols, Yinfei Yang, and Zhe Gan. Ferret-ui: Grounded mobile ui understanding with multimodal llms, 2024.
- [Zhang *et al.*, 2022] Ziqian Zhang, Yulei Liu, Shengcheng Yu, Xin Li, Yexiao Yun, Chunrong Fang, and Zhenyu Chen. Unirltest: universal platform-independent testing with reinforcement learning via image understanding. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2022*, page 805–808, New York, NY, USA, 2022. Association for Computing Machinery.
- [Zhao *et al.*, 2023] Yudi Zhao, Yuan Zhang, and Min Yang. Remote code execution from SSTI in the sandbox: Automatically detecting and exploiting template escape bugs. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 3691–3708, Anaheim, CA, August 2023. USENIX Association.
- [Zheng *et al.*, 2021] Yan Zheng, Yi Liu, Xiaofei Xie, Yepang Liu, Lei Ma, Jianye Hao, and Yang Liu. Automatic web testing using curiosity-driven reinforcement learning. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pages 423–435, 2021.
- [Zheng *et al.*, 2024] Xi Zheng, Aloysius K. Mok, Ruzica Piskac, Yong Jae Lee, Bhaskar Krishnamachari, Dakai Zhu, Oleg Sokolsky, and Insup Lee. Testing learning-enabled cyber-physical systems with large-language models: A formal approach. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering, FSE 2024*, page 467–471, New York, NY, USA, 2024. Association for Computing Machinery.